

Стандарти виконання завдань курсу

ProfiTsoft Full Stack Developer

Вимоги до виконання завдань

1. Код виконаного завдання має бути розміщений в особистому github або gitlab репозиторії інтерна. В класрумі на сторінці завдання потрібно додати посилання на відповідний репозиторій і натиснути “Відмітити як виконане”.
2. Код в репозиторії має бути розміщений стандартним чином для maven-проектів. Мається на увазі відповідна структура папок, розміщення вихідних кодів, ресурсів, тестів, тощо. У репозиторії має бути присутнім файл .gitignore, який виключає додавання “зайвих” файлів і папок (результатів білду, службових файлів IDE). У pom.xml повинні бути присутніми всі необхідні залежності. Коли куратор клонує проект на свій комп'ютер для перевірки, проект повинен “білдитися” без будь-яких зайвих рухів (додавання залежностей, тощо) з боку куратора. Проект має бути розміщений в корні git репозиторію. Будь-ласка, не створюйте в одному репозиторії декілька папок з проектами.
3. Код повинен мати зрозумілу точку запуску. Це або метод main, або unit-тести (якщо завдання вимагає написати лише певну функцію та покриття тестами для неї). Якщо завдання передбачає більш складні кроки для запуску, аніж запуск тестів або метода main, інструкції по збірці та запуску повинні бути вказані в файлі README.md в корні репозиторію. В ідеалі, куратор клонує проект на свій комп'ютер, запускає код, і все відпрацьовує.
4. В коді повинна використовуватися Java версії 21. Її можна скачати тут: <https://jdk.java.net/archive/>.
5. Виконане завдання повинно враховувати всі вимоги, вказані явно в описі завдання (перечитуйте його ще раз перед відправкою). Вимоги можуть бути функціональними (яку логіку повинна реалізовувати програма), так і технічні (про використання певних технологій/бібліотек/структур даних, тощо). Якщо технічні вимоги явно не вказані, то вибір технологій/бібліотек/структур даних залишається на ваш розсуд, але він має бути у тренді того матеріалу, який ми проходимо.
6. Наявність unit-тестів обов'язкова лише в тих завданнях, де це вказано явно в описі. Там, де не вказано явно, наявність unit-тестів не обов'язкова, за їх відсутність бали зніматися не будуть. Але якщо тести все ж таки будуть присутні і в цих завданнях, це може бути “плюсом” (частково компенсувати якісь інші недоліки, якщо вони будуть). Тому що гарне тестове покриття - це завжди добре :-).
7. Java-код має відповідати стандартам іменування класів, змінних, методів, констант, пакетів, тощо (стандарти іменування зазначаються в першій лекції). В проекті має

використовуватися єдиний стиль форматування коду (відступи 2 або 4 пробіли, або таби, тут на ваш вибір, але щоб всюди було однаково).

Інші аспекти, на які буде звертатися увага під час перевірки

1. Структура коду. Відповідність назв класів, методів і змінних їх функціоналу. В ідеалі, методи мають реалізовувати одну певну функцію на відповідному рівні абстракції. Уникайте занадто довгих і складних методів, зі складними умовними конструкціями і кількома рівнями вкладеності циклів. Якщо відповідної складності вимагає логіка, виносьте логічно завершені фрагменти коду у приватні методи з відповідними назвами, навіть якщо ці методи викликаються лише в одному місці. Код лямбди, якщо він не є однією строкою, зазвичай можна винести в метод, і перетворити на одну строку.

2. Оптимальність коду. Наприклад:

- довго виконувані операції в середині циклу, які там не потрібні
- незакриття ресурсів
- занадто складні та незрозумілі алгоритми
- невірна обробка помилок (вирогідність виникнення `NullPointerException`, тощо)
- фрагменти коду, що дублюються
- фрагменти коду, що ніде не використовуються
- і т.і.

3. Повнота покриття логіки unit-тестами: врахування не лише очікуваних сценаріїв, а також і негативних. Правильна структура unit-тестів: атомарність, незалежність, дотримання структури `given-when-then`.

4. Простота. Ми цінуємо "простоту". Простий код, як правило, легше читати, тестувати та обслуговувати. Це сприяє швидшій розробці та меншій кількості помилок. Ієрархії класів та використання паттернів корисні тоді, коли вони вирішують конкретну задачу, яку можна чітко сформулювати. Наприклад, ієрархія класів має сенс, якщо ви можете перевикористати поля або логіку суперкласу в кількох класах-нащадках, або коли ви створюєте компонент, який працює з більш абстрактною сутністю і не повинен знати про ваш бізнес-домен.

Якщо у суперкласу є лише один клас-нащадок, і окремо він ніде не використовується, варто подумати над їх об'єднанням в один клас. Це відповідає принципу [Бритви Оккама](#): не "плідить зайві сутності". Використовуйте принципи [YAGNI](#) ("You Aren't Gonna Need It") та [KISS](#) ("Keep It Simple, Stupid").

Виділення логічних частин коду в окремі класи, що реалізують окремі шари в N-tier архітектурі або вирішують конкретні задачі згідно з принципом єдиної відповідальності (наприклад, `XxxParser`, `XxxValidator`, `XxxConverter`), може додавати простоти, роблячи інші класи більш зрозумілими.

Принципи структуризації коду можуть бути частиною стандартів проекту або індивідуального стилю розробника. Під час перевірки завдань ми звертатимемо увагу на структуризацію і надаватимемо зауваження, якщо її можна покращити. Оцінка може знижуватися лише в разі, якщо недостатньо гарна структуризація матиме негативні наслідки, які будуть зазначені в зауваженнях.